scientific reports

OPEN

Improved double DQN with deep reinforcement learning for UAV indoor autonomous obstacle avoidance

Ruiqi Yu[1], Qingdang Li[2], Jiewei Ji[1], Tingting Wu[1], Jian Mao[3], Shun Liu[4] & Zhen Sun[1]✉

Aiming at the problems of insufficient autonomous obstacle avoidance performance of UAVs in complex indoor environments, an improved Double DQN algorithm based on deep reinforcement learning is proposed. The algorithm enhances the perception and learning capabilities by optimizing the network model and employs a dynamic exploration strategy that encourages exploration in the early stage and reduces it later to accelerate convergence and improve efficiency. Simulation experiments in two scenarios of varying complexity, using an indoor simulation environment built with AirSim and UE4(Unreal Engine 4), show that in the simpler scenario, the average cumulative reward increased by 22.88%, the maximum reward increased by 101.56%, the average safe flight distance increased by 23.17%, and the maximum safe flight distance by 105.62%. In the more complex scenario, the average cumulative reward increased by 2.66%, the maximum reward increased by 88.77%, the average safe flight distance increased by 2.05%, and the maximum safe flight distance by 84.68%.

Keywords UAV, Indoor autonomous obstacle avoidance, Deep reinforcement learning, Double DQN algorithm, Dynamic exploration strategy

With the rapid development of technology, Unmanned Aerial Vehicle (UAV) technology is increasingly applied in various fields such as agricultural monitoring, urban logistics, and disaster relief[1,2]. UAVs have become an important tool in modern society due to their flexibility and efficiency. Among these applications, autonomous obstacle avoidance is one of the key technologies for the safe and efficient operation of UAVs. Excellent autonomous obstacle avoidance capabilities can ensure that UAVs avoid obstacles in complex environments, ensuring flight safety and improving mission execution efficiency. However, achieving efficient autonomous obstacle avoidance in complex environments remains a major challenge in the development of UAV technology[3].

Traditional obstacle avoidance methods, such as the A* algorithm[4,5], Rapidly-exploring Random Tree (RRT) algorithm[6,7], artificial potential field method[8], and Particle Swarm Optimization (PSO) algorithm[9], can achieve obstacle avoidance to a certain extent. Mochammad Sahal et al.[10] proposed a cooperative formation control and obstacle avoidance method for multi-UAVs based on guidance route and artificial potential field (APF) methods. By integrating a fuzzy controller, guidance route, and optimized APF methods, efficient obstacle avoidance for multi-UAVs in complex environments was achieved. In addition, Mochammad Sahal et al.[11] further combined gradient consensus with the APF method to address the obstacle avoidance problem for multi-agents tracking a moving source, demonstrating the effectiveness of the method in dealing with both static and dynamic obstacles. However, these traditional methods still have limitations in dynamic and complex environments, mainly manifested in high computational costs, poor real-time performance, and insufficient path smoothness[12], which fail to meet the demands of efficient autonomous obstacle avoidance for UAVs.

Reinforcement learning has become increasingly popular in the field of UAV autonomous obstacle avoidance[13]. In recent years, with the introduction of deep learning, an increasing number of researchers have chosen to use deep reinforcement learning for the study of UAV autonomous obstacle avoidance algorithms. The advent of deep reinforcement learning can be traced back to DeepMind's integration of Q-Learning with Convolutional Neural Networks (CNNs) in 2013, which led to the development of the Deep Q-Network

[1]College of Data Science, Qingdao University of Science and Technology, Qingdao 266061, China. [2]College of Sino-German Science and Technology, Qingdao University of Science and Technology, Qingdao 266061, China. [3]Qingdao Central Hospital, University of Health and Rehabilitation Sciences, Qingdao 266000, China. [4]College of Information Science and Technology, Qingdao University of Science and Technology, Qingdao 266061, China. ✉email: sunzhen@qust.edu.cn

(DQN). By leveraging deep neural networks to approximate the Q-function, DQN effectively addressed the issue of dimensionality curse that plagues traditional Q-Learning in high-dimensional state spaces[14]. Yao et al.[15] improved the greedy strategy, reward function, value iteration method, and sampling probability of DQN, enabling the algorithm to achieve better convergence, higher returns, and a more stable training process in path planning. Li Ming et al.[16] improved the exploration factor of the traditional DQN algorithm for robot path planning in complex desert environments, allowing it to change with the robot's understanding of the environment and set a dynamic reward function, which increased the algorithm's convergence speed and search efficiency[17]. Double DQN, proposed by DeepMind in 2016, further optimized DQN by introducing two independent Q-networks to address the overestimation problem in DQN. In Double DQN, one network is used to select the optimal action, while the other estimates the Q-value of that action, more accurately reflecting the action value of the current policy and reducing estimation bias in target calculation[18]. Tang et al.[19] addressed the sparse reward and overestimation problems of traditional Double DQN by introducing dynamically weighted high-quality experiences and integrating prior knowledge from Double DQN and Average Double DQN, thereby improving the performance of path planning for unmanned ground vehicles in complex 3D environments. Yao et al.[20] improved the Double DQN network structure by introducing a Gated Recurrent Unit (GRU) to handle temporal information and enhance performance, combined with prioritized experience replay to accelerate network fusion, thus increasing the training speed and obstacle avoidance performance of UAVs. Abhilasha et al.[21] enhanced the obstacle avoidance performance of robot manipulators by combining Double DQN with bionic algorithms. Mochammad Sahal et al.[22] proposed an obstacle avoidance system for autonomous vehicles based on D3QN. By integrating the Dueling and Double-Q mechanisms, D3QN can more effectively handle obstacle avoidance problems in complex environments. The method introduces the Dueling mechanism to separate state values and action advantage values, thereby more accurately assessing the value of actions, while utilizing the Double-Q mechanism to reduce the problem of overestimation. However, traditional Double DQN still has some shortcomings in complex environments, such as limited perception and learning capabilities for environmental states and inflexible exploration strategies, leading to slow algorithm convergence and low obstacle avoidance efficiency.

To tackle these challenges, this paper proposes an improved version of Double DQN, building upon the conventional Double DQN framework. The key innovations and contributions of this work are as follow:

1. Long Short-Term Memory (LSTM) networks[23] and noise layers[24] are introduced into the network model. The LSTM enhances the adaptation to complex environments and the robustness of the policy[25], while the noise layer increases exploration diversity by introducing noise into the neural network weights, helping UAVs explore the environment more extensively in the early stages of training.
2. A prioritized experience replay module based on mean squared error[26] is introduced to prioritize experiences that are more beneficial for model learning, thereby improving learning efficiency and accelerating convergence.
3. A dynamic exploration rate adjustment strategy is designed to encourage exploration in the early stages and reduce exploration in the later stages to utilize knowledge, thereby accelerating algorithm convergence and improving obstacle avoidance efficiency.

Preliminaries on reinforcement learning

The concept of reinforcement learning

Reinforcement learning, a crucial subset of machine learning, focuses on how an agent can optimize the rewards it receives from its interactions within an environment. This learning paradigm is composed of two fundamental elements: the agent and the environment. Throughout the reinforcement learning process, the agent engages in continuous interaction with the environment. Specifically, the agent chooses an action based on the current state, performs this action within the environment, and subsequently receives a reward. The environment then transitions to the next state[27]. The overarching objective of the agent is to maximize cumulative rewards. The foundational structure of reinforcement learning is depicted in Fig. 1.

Q-learning algorithm

The Q-learning algorithm, a seminal reinforcement learning method, centers on guiding an agent to make optimal decisions by constructing a Q-Table[28]. This table is two-dimensional, with rows corresponding to states s and columns to actions a. Each entry $Q(s,a)$ signifies the anticipated return when action a is executed in state s[29,30]. These Q-values are incrementally learned and refined through ongoing interactions with the environment. The update mechanism is illustrated in Eq. (1):

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{1}$$

where α is the learning rate ($0 < \alpha < 1$), which controls the extent to which new information updates old information; r is the immediate reward for the current action; γ is the discount factor ($0 < \gamma < 1$), which measures the current value of future rewards; and $\max_{\alpha'} Q(s',a')$ is the maximum Q-value of all potential actions in the subsequent state s′ , reflecting the estimate of the optimal future return.

DQN algorithm

To address the limitations of Q-learning in handling complex and high-dimensional state spaces, such as the high storage cost of Q-tables and the difficulty in dealing with continuous state spaces, as well as the issue of overestimation in Q-value updates, the DeepMind team proposed the DQN algorithm in 2013. This algorithm integrates deep learning and reinforcement learning by employing a deep neural network, the structure of

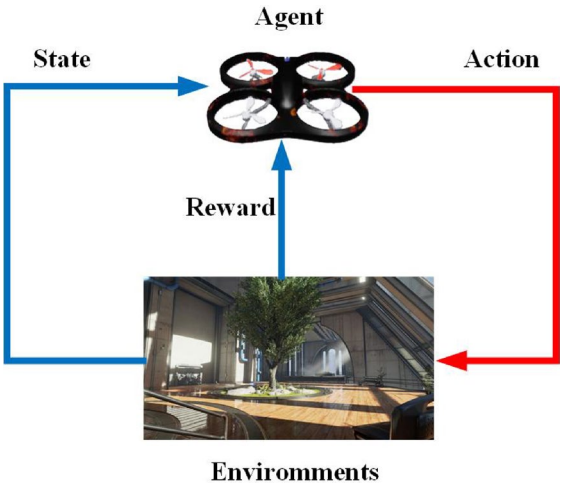


Fig. 1. Basic framework of reinforcement learning.

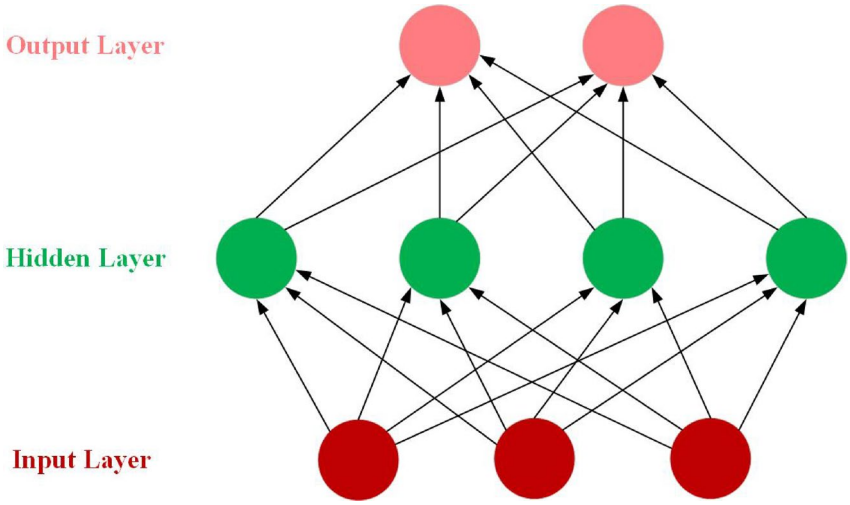


Fig. 2. Neural network structure.

which is shown in Fig. 2, to approximate the Q-value function, effectively resolving the limitations of traditional Q-learning.

DQN uses a deep neural network $Q(s,a;\theta)$ to approximate the Q-function. Its goal is to learn the mapping from state-action pairs to Q-values[31]. Equation (2) shows the Q-value approximation formula of DQN:

$$Q(s,a;\theta) \approx Q^*(s,a) \tag{2}$$

where $Q^*(s,a)$ is the true Q-value, and θ is the parameter of the deep neural network.

DQN processes state information using a deep neural network. The environment state is used as the input to the deep neural network. The network processes the state information through forward propagation and outputs a one-dimensional vector, where each element of the vector represents the Q-value of the corresponding action. Figure 3 shows the framework of DQN algorithm.

The DQN algorithm mainly consists of two parts: executing actions and training the network[32]. In the execution phase, the agent feeeds the state s into the Q-network to retrieve the Q-values of all possible actions. It then chooses an action using the ε-greedy strategy: with a probability of ε ($0<\varepsilon<1$), the action with the highest Q-value is selected; otherwise, with a probability of $1-\varepsilon$, an action is selected randomly. After selecting the action a, it is executed in the environment, yielding a new state s' and a reward r. This experience tuple (s,a,r,s') is subsequently stored in the experience replay buffer. In the training phase, the algorithm randomly samples a batch of experiences from the experience replay pool as training samples. The original state s and action a are fed into the Q-network to obtain the predicted Q-value. The new state s′ is input into the target network to determine the maximum Q-value, which is combined with the reward to compute the target Q-value. The

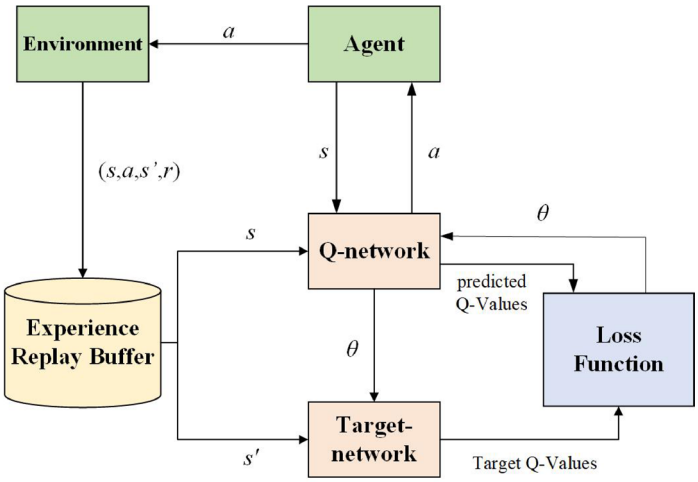


Fig. 3. DQN algorithm framework.

parameters of Q-network are updated via backpropagation using the mean squared error loss function[33,34]. Equation (3) illustrates the calculation of the target Q-value:

$$Q_{T\arg et} = r + \gamma \max_{a'} Q(s', a'; \theta^{-}) \tag{3}$$

where r represents the immediate reward for the current action, γ is the discount factor ($0 < \gamma < 1$), which weighs the significance of immediate and future rewards, θ denotes the parameter of the Q-network, and θ^{-} is the parameter of the target network. The parameters of the target network are copied from the Q-network every N time steps to maintain stability. Typically, the algorithm accumulates experiences over multiple time steps before conducting a round of training. Equation (4) shows the process of calculating the loss function in DQN.

$$L(\theta) = E_{(s,a,s',r)\sim D}[(Q_{T\arg et} - Q(s, a; \theta))^2] \tag{4}$$

where D is the experience replay buffer, and $E_{(s,a,s',r)\sim D}$ denotes the expectation over samples in the experience replay buffer D.

Double DQN algorithm

The traditional DQN algorithm tends to overestimate Q-values because it uses the same network to select the optimal action and estimate the Q-value of that action which leads to an unstable learning process. To address this issue, the Double DQN algorithm was proposed. The core idea of Double DQN is to separate action selection and Q-value estimation by introducing two independent neural networks to reduce the overestimation of Q-values[35]. These two independent neural networks are the evaluation network also known as the Q-network, and the target network. The evaluation network is responsible for selecting the optimal action, whereas the target network is utilized to estimate the Q-value associated with that action. The procedure for updating network parameters is illustrated in Fig. 4.

In the training phase, the Double DQN algorithm samples a batch of experiences from the experience replay pool as training samples. The original state s and action a are fed into the evaluation network to compute the predicted Q-value $Q(s,a;\theta)$. Then, the new state s′ is input into the evaluation network to identify the action $a^{\star}$ that yields the maximum Q-value. Subsequently, the new state s′ and the selected action $a^{\star}$ are input into the target network to determine the target Q-value[36,37], as depicted in Eq. (5):

$$Q_{T\arg et} = r + \gamma Q(s', \arg\max_{a'} Q(s', a*; \theta); \theta^{-}) \tag{5}$$

where r is the immediate reward for the current action, γ is the discount factor ($0 < \gamma < 1$), which balances the importance of immediate and future rewards, $\arg\max_{a'} Q(s', a*; \theta)$ is the action with the maximum Q-value selected by the evaluation network for the new state s′, θ is the parameter of the evaluation network, and θ^{-} is the parameter of the target network. The parameters of the target network are copied from the evaluation network every N time steps to maintain stability[38,39]. Typically, the algorithm accumulates experiences over multiple time steps before conducting a round of training. The process of calculating the loss function in Double DQN is the same as in DQN, as shown in Eq. (4).

Improved double DQN algorithm

Network model design

To elevate the autonomous obstacle avoidance efficiency of UAVs in complex indoor environments, this paper optimizes Double DQN's netwoek model. First, the Long Short-Term Memory (LSTM) network is incorporated

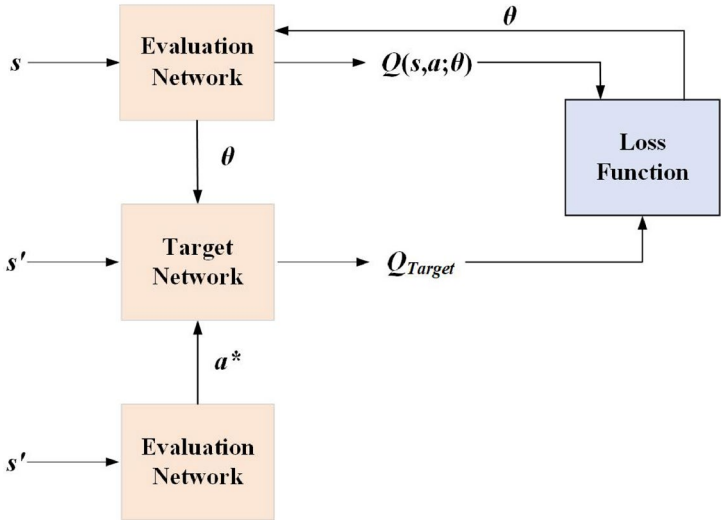


Fig. 4. Network parameter update of Double DQN.

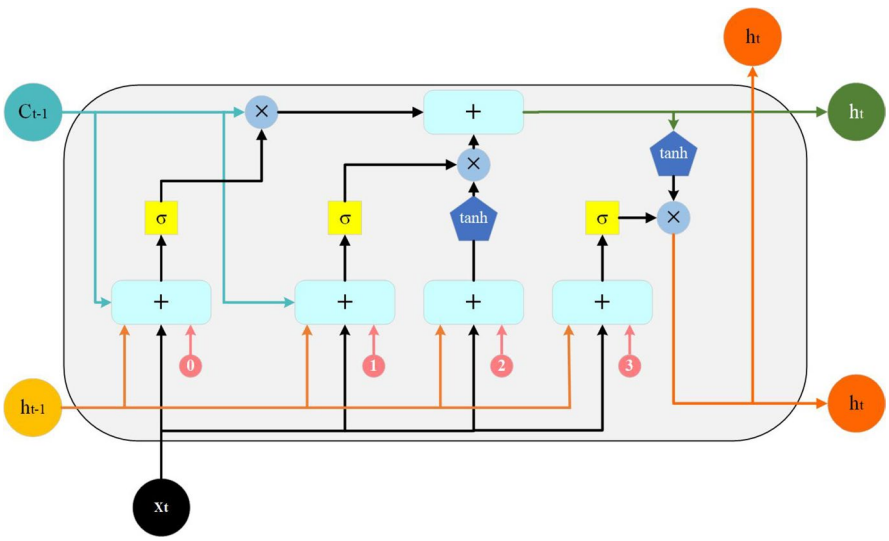


Fig. 5. LSTM structure diagram.

to improve the model's adaptability and robustness in complex environments. As a specialized form of recurrent neural network, LSTM is capable of effectively managing long-term dependencies within time-series data. Through its gating mechanism, LSTM avoids the gradient vanishing or exploding problems that are common in traditional RNNs. In the model, the LSTM layer receives feature sequences extracted by the convolutional neural network and outputs features processed with temporal dependencies. This helps the UAV perceive the location of obstacles in real-time and make decisions accordingly. The LSTM structure is shown in Fig. 5. Second, to increase the diversity of exploration, a noise layer is added to the neural network. The noise layer introduces Gaussian noise during the initial phase of training, increasing the randomness of exploration. This helps the UAV explore the environment more extensively and avoid falling into local optima. This enhancement not only accelerates the algorithm's convergence rate but also bolsters the model's capacity for generalization in complex environments.

The overall architecture of the network model consists of a convolutional backbone, a noise injection module, an LSTM module, and fully connected layers. Figure 6 shows the network model structure. The input to the network model is the RGB image captured by the UAV's camera, which serves as the state input for the deep reinforcement learning model. The convolutional backbone is composed of three convolutional layers, which are used to extract spatial features from the input images. The first convolutional layer has a kernel size of 7×7, with 3 input channels and 96 output channels, and a stride of 4. The second convolutional layer has a kernel size of 5×5, with 96 input channels and 64 output channels, and a stride of 1. The third convolutional layer has a kernel size of 3×3, with 64 input channels and 64 output channels, and a stride of 1. Max-pooling layers are added after the

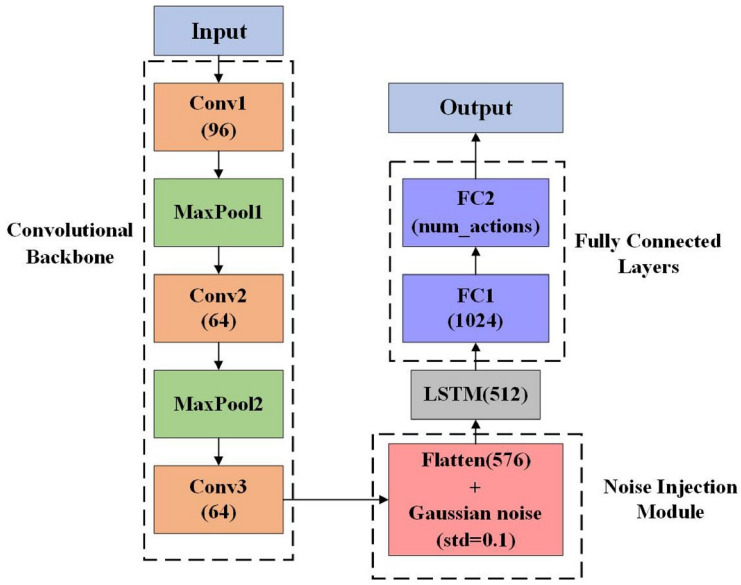


Fig. 6. Network model structure diagram.

first two convolutional layers to gradually reduce the spatial dimensions of the feature maps while retaining key features. The ReLU activation function serves to incorporate non-linearity within the network.

Following the output of the convolutional backbone, a noise injection module is added. By introducing Gaussian noise at the feature level, this module increases the randomness of exploration, helping the UAV to explore the environment more extensively and avoid falling into local optima. In this paper, the standard deviation of the Gaussian noise is set to 0.1.

The LSTM layer receives the feature sequences extracted by the convolutional backbone and processes the long-term dependencies in the time-series data. Through its gating mechanisms (input gate, forget gate, and output gate), LSTM controls the flow of information, avoiding the gradient vanishing or exploding problems that are common in traditional RNNs. In this paper's model, the LSTM layer has 512 units.

Finally, the fully connected layers transform the output of the LSTM layer into a one-dimensional feature vector, which represents the Q-values of all actions in the action space A. The first fully connected layer has 512 input units and 1024 output units, with the ReLU activation function. The second fully connected layer has 1024 input units and num_actions (number of actions) output units, with a linear activation function.

Dynamic exploration strategy

In reinforcement learning, the ε-greedy strategy is a common exploration mechanism. The exploration rate ε ($0 < \varepsilon < 1$) selects the action with the highest Q-value with a probability of ε, representing the UAV's exploitation of the known optimal policy to execute tasks. With a probability of 1-ε, the action is selected randomly, indicating the UAV's exploration of new unknown areas. However, this strategy may lead to insufficient exploration in the early stages and excessive exploration in the later stages, affecting the convergence speed and performance of the algorithm. To balance the relationship between exploration and exploitation, a fixed growth exploration rate is a common strategy. In this strategy, the exploration rate ε gradually increases linearly from an initial value to a larger value as the number of iterations increases. However, in complex or dynamically changing environments, a fixed exploration rate growth method may not adapt to environmental changes, leading the algorithm to suboptimal solutions.

To tackle these challenges, this paper proposes an improved dynamic exploration strategy aimed at dynamically adjusting the exploration rate ε according to the training progress to achieve the goal of thorough exploration in the early stages and stable exploitation of known knowledge in the later stages. In the strategy, the minimum exploration rate ε_{min} and the maximum exploration rate ε_{max} are set. When the exploration rate ε is less than or equal to the minimum exploration rate ε_{min}, the exploration rate directly uses ε_{min}. When the exploration rate ε is greater than or equal to the maximum exploration rate ε_{max}, the exploration rate directly uses ε_{max}. This strategy is implemented through Eq. (6):

$$\varepsilon = \begin{cases} \varepsilon_{min}, & \varepsilon \leqslant \varepsilon_{min} \\ \varepsilon_{min} + (\varepsilon_{max} - \varepsilon_{min}) \times (1 - e^{-\alpha(\frac{t-w}{b-w})^{\beta}}), & \varepsilon_{min} < \varepsilon < \varepsilon_{max} \\ \varepsilon_{max}, & \varepsilon \geqslant \varepsilon_{max} \end{cases} \tag{6}$$

where t is the current iteration number; b is the saturation parameter, which is smaller than the total number of iterations and defines the inflection point where the exploration rate transitions from rapid growth to gradual saturation. After t exceeds b, the rate of increase will slow down and eventually stabilize; w is the number of

iterations to wait before training begins, which is the number of explorations before training starts; $\alpha(\alpha > 0)$ and $\beta(\beta > 0)$ are parameters that control the growth rate and shape of the exploration rate.

At the oneset of training, when the iteration number t is close to w, the exploration rate ε is close to ε_{min}, encouraging more random exploration. As the iteration number t increases, the growth rate of the exploration rate accelerates to balance exploration and exploitation. In the later stages of training, after the iteration number t exceeds the saturation parameter b, the rate of increase will slow down and the exploration rate ε will approach ε_{max}, gradually stabilizing. During this phase, the UAV will reduce exploration and increasingly rely on the knowledge it has learned.

Prioritized experience replay

In the traditional Double DQN algorithm, the experience replay mechanism trains by randomly sampling from the experience replay buffer. Although this method reduces the temporal correlation of data and improves the reuse of samples, it fails to fully leverage the importance of each sample. To further enhance learning efficiency and accelerate convergence, this paper introduces the Prioritized Experience Replay (PER) mechanism.

The core idea of Prioritized Experience Replay is to rank samples based on their importance and prioritize sampling experiences that are more beneficial to model learning. The importance of a sample is typically measured by its Temporal Difference (TD) error. Samples with larger TD errors are considered more helpful for model updates. Therefore, the PER mechanism assigns a priority weight pi to each sample and samples based on these weights. Equation (7) shows the sampling probability:

$$P(i) = \frac{p_i^{\alpha}}{\sum_{k=1}^{N} \mathrm{p}_k^{\alpha}} \tag{7}$$

Where N represents the size of the experience replay buffer, and α is a hyperparameter used to regulate the level of prioritization. When $\alpha = 0$, all samples have the same priority, equivalent to random sampling. When α approaches 1, the impact of priority is maximized, and sampling tends to favor samples with higher priorities.

In addition to this, to ensure fairness in the sampling process and prevent high-priority samples from being over-sampled, the PER mechanism introduces Importance Sampling Weights (ISW). The purpose of ISW is to correct the sampling probabilities so that each sample contributes more evenly to the gradient update. Equation (8) shows the calculation formula for ISW:

$$\omega_i = \left(\frac{1}{N} \cdot \frac{1}{\mathrm{p}_i}\right)^{\beta} \tag{8}$$

where N represents the size of the experience replay buffer, and β is a hyperparameter that adjusts the degree of bias correction. When $\beta = 0$, the ω_i is 1, equivalent to no importance sampling. When β approaches 1, the impact of ω_i is maximized, and sampling tends to favor samples with higher priorities.

Overall framework of the improved double DQN algorithm

Figure 7 illustrates the overall structure of the improved Double DQN algorithm. Initially, the network and the UAV's state are set up. During the execution phase, the UAV feeds the state s into the evaluation network to retrieve the Q-values of all possible actions and chooses an action based on the dynamic exploration strategy. After executing the chosen action a in the environment, the UAV receives a new state s′ and a reward r. The experience (s,a,r,s') is stored in the experience replay pool. The UAV continues to interact with the environment over multiple time steps before proceeding to the training phase.

In the training phase, the algorithm samples training data from the experience replay pool using prioritized experience replay. The original state s and action a are input into the evaluation network to generate the

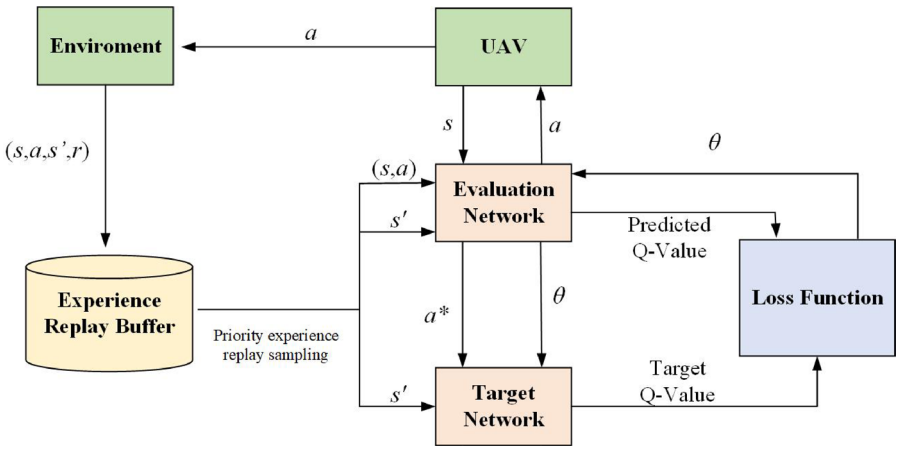


Fig. 7. Improved Double DQN overall framework.

predicted Q-value. Meanwhile, the new state s' is fed into the evaluation network to identify the action a^* with the highest Q-value. The state s′ and action a^* are then input into the target network to compute the target Q-value, incorporating the reward. The evaluation network's parameters are updated via backpropagation using the mean squared error loss function. To maintain stability, the target network's parameters are periodically updated by copying them from the evaluation network every N time steps.

Simulation and analysis

To verify the effectiveness of the improved Double DQN algorithm in UAV indoor autonomous obstacle avoidance, this paper constructs a high-resolution and high-performance simulation environment based on AirSim and UE4 (Unreal Engine 4). The simulation environment includes various indoor scenarios, such as rooms with different layouts, corridors, and environments with various static obstacles. Figure 8 shows the framework of the simulation environment.

During the experimental process, to ensure the efficient operation of the simulation experiments and the reliability of the results, the hardware configuration of the experimental environment used in this paper is as follows: the operating system is Windows 11, the processor is an Intel (R) Core (TM) i9-10900X CPU @ 3.70 GHz, equipped with 64GB of memory, and the graphics card is an NVIDIA GeForce RTX 3090 with 24GB of video memory.

In this paper, we design a reward function based on depth information to guide the UAV to autonomously avoid obstacles and fly safely in complex indoor environments. Specifically, at each time step, we obtain a depth map captured by the camera in the simulation environment and calculate the average depth value of the central region. The detailed steps are as follows:

1. Obtain the depth map D from the simulation environment. Apply thresholding to set depth values exceeding the predefined threshold *thresh* to *thresh*,which is shown as Eq. (9):

$$D = min(D, thresh) \tag{9}$$

Then normalize the resulting depth map D, Eq. (10) shows the normalization process:

$$D_{norm} = \frac{D}{thresh} \tag{10}$$

2. Based on the dimensions $H \times W$ of the normalized depth map D_{norm}, determine the boundaries of the central region. Calculate the start and end coordinates of the central region to ensure it covers the center of the depth map. Assume the width of the central region is w and its height is h. The start coordinates are $\left(\frac{W-w}{2}, \frac{H-h}{2}\right)$, and the end coordinates are $\left(\frac{W+w}{2}, \frac{H+h}{2}\right)$.
3. Extract all pixel values from the central region of the normalized depth map D_{norm} to form an array D_{mid} . Equation (11) shows the extraction process:

$$D_{mid} = D_{norm}\left[(\frac{H-h}{2}):(\frac{H+h}{2}),(\frac{W-w}{2}):(\frac{W+w}{2})\right] \tag{11}$$

4. Sort the pixel values in D_{mid} and remove a certain proportion of outliers(e.g., maximum and minimum values) to reduce their impact on the average depth value. Assume the proportion of outliers is λ, The range of retained pixel values is: $[\lambda \times len(D_{mid}), (1-\lambda) \times len(D_{mid})]$, where $len(D_{mid})$ represents the total number of elements in the array D_{mid}. Then calculate the average depth value of the remaining pixel values to obtain the average depth value of the central region D_{new}. Equation (12) shows the process:

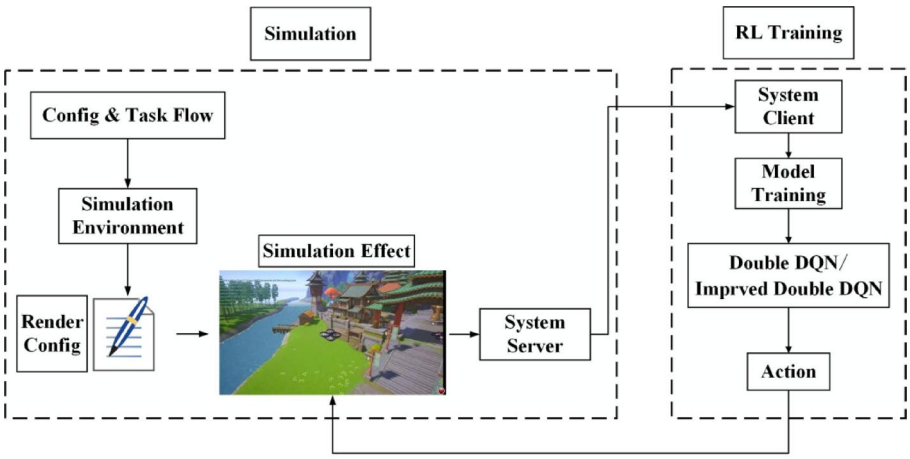


Fig. 8. Simulation environment framework.

$$D_{new} = \frac{\sum_{i=\lambda \times len(D_{mid})}^{(1-\lambda)\times len(D_{mid})} D_{mid}[i]}{\text{len}(D_{mid}) \times (1-2\lambda)} \tag{12}$$

After obtaining the average depth value D_{new} of the central region, if D_{new} is less than the predefined collision threshold *crash_threshold*, it is considered that the drone has collided, and the reward value is set to −1, ending the current episode. Otherwise, the drone is considered to be flying safely, and the reward value is equal to the average depth value D_{new}. Equation (13) shows the reward calculation process:

$$reward = \begin{cases} -1, & D_{new} < crash_threshold \\ D_{new}, & D_{new} \geqslant crash_threshold \end{cases} \tag{13}$$

The hyperparameter configuration of the improved Double DQN algorithm proposed in this paper is shown in Table 1. To ensure consistency in the experiments, except for the hyperparameters specific to the dynamic exploration strategy and the prioritized replay mechanism, all other hyperparameters are kept consistent with the original algorithm.

To evaluate the performance of the traditional Double DQN algorithm and the improved Double DQN algorithm proposed in this paper for UAV indoor autonomous obstacle avoidance, the experiments were designed to be conducted in two indoor environments with different levels of complexity, with each environment undergoing 150,000 iterations using both the traditional and improved Double DQN algorithms. : The two indoor environments with different complexities are referred to as the "Long Corridor" and "Compact Living Room".

The main metrics used in this paper include memory usage, average cumulative reward, maximum cumulative reward, average safe flight distance, and maximum safe flight distance. Cumulative reward refers to the sum of all reward values over all time steps within each episode; flight distance refers to the distance the drone travels within each episode.

To more accurately assess the performance of the algorithms in their stable states, this paper selects data from the episode where the reward values and flight distances begin to significantly increase to the last episode for calculating the average cumulative reward and average flight distance. This is because in the early stages of training, the behavior of the drone may be highly unstable, leading to large fluctuations in reward values and flight distances. By disregarding the early unstable phase, a more accurate assessment of the algorithms' performance in their respective stable states can be made. Additionally, to minimize the impact of extreme values on the statistical results, this paper removes 5% of the maximum values and 5% of the minimum values when calculating the averages. Equation (14) shows the calculation of the average cumulative reward:

$$R_{\text{ave}} = \frac{1}{N_e} \sum_{i \in I} R_i \tag{14}$$

where N_e represents the number of valid episodes after removing the top 5% and bottom 5% of value, I denotes the set of valid episodes, and R_i is the cumulative reward of the i-th episode. Equation (15) shows the calculation of the average flight distance:

$$D_{ave} = \frac{1}{N_e} \sum_{i \in I} D_i \tag{15}$$

where N_e represents the number of valid episodes after removing the top 5% and bottom 5% of value, I denotes the set of valid episodes, and D_i is the safe flight distance of the i-th episode. The maximum cumulative reward is the highest cumulative reward value among all episodes, as shown in Eq. (16):

$$R_{\max} = \max\{R_i | i = 1, 2, \cdots, N\} \tag{16}$$

where N is the total number of episodes. Similarly, the farthest flight distance is the highest flight distance value among all episodes, as shown in Eq. (17):

Hyperparameters	Value	Hyperparameters	Value
State space	103	Depth threshold	50
Action space	25	Outlier ratio	0.05
Pre-training iterations	5000	Max exploration rate	0.95
Total iterations	15×10^4	Min exploration rate	0.001
Saturation iterations	10×10^4	Priority factor	0.6
Collision threshold	1.3	Sampling coefficient	0.4
Discount factor	0.99	Exploration growth rate	2
Learning rate	2×10^{-6}	Exploration shape parameter	2

Table 1. Algorithm hyperparameter configuration.

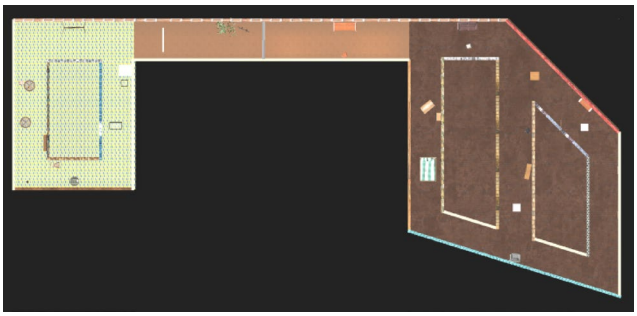


Fig. 9. The environment map of "Long Corridor".

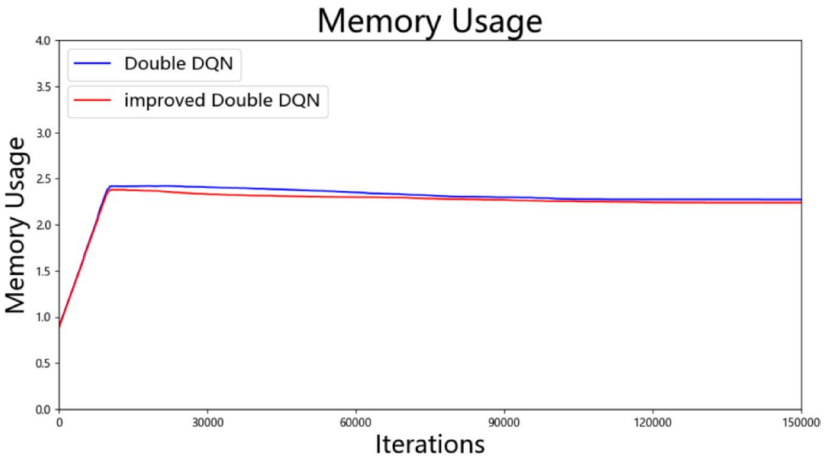


Fig. 10. Comparison of memory usage in "Long Corridor".

$$D_{\max} = \max\{D_i | i = 1, 2, \cdots, N\} \tag{17}$$

where N is the total number of episodes.

Comparative experiment in "long corridor"

The environment is shown in Fig. 9, characterized by a long corridor with a relatively simple and straightforward path that has few obstacles. In this experiment, the UAV's starting position is set at (994, 95, 220), with an initial yaw angle of 21 degrees, and both roll and pitch angles set to 0 degrees.

Following 150,000 iterations in this experimental environment, utilizing both the improved Double DQN algorithm and the traditional Double DQN algorithm, the training outcomes are depicted in Figs. 10, 11 and 12. Figure 10 illustrates the memory usage of the improved and traditional Double DQN algorithms during the 150,000 iterations in the simulation environment, indicating that the improved algorithm is slightly more memory-efficient. Figures 11 and 12 depict the changes in cumulative rewards and flight distances for both algorithms throughout the experiment. It is evident from the figures that the improved algorithm has higher cumulative rewards and safe flight distances than the traditional algorithm for most of the training period, and it shows a faster increase in rewards in the later stages. Both models start to significantly improve in reward values and safe flight distances after about 4,000 episodes. To accurately reflect the algorithms' performance in stable states and avoid the impact of early instability, we calculated the average cumulative rewards and flight distances from episode 4,000 to the last episode, excluding the top 5% and bottom 5% of values to mitigate the influence of outliers. The experimental results, shown in Table 2, reveal that the improved algorithm improved the average cumulative reward by 22.88%, the maximum cumulative reward by 101.56%, the average safe flight distance by 23.17%, and the maximum safe flight distance by 105.62% compared to the traditional Double DQN algorithm. These findings demonstrate that the improved Double DQN algorithm significantly outperforms the traditional one in terms of cumulative rewards and flight distances during stable phases, indicating superior performance and stability in the experimental environment.

Comparative experiment in "compact living room"

The environment is shown in Fig. 13, characterized by a compact layout with multiple rooms and pieces of furniture, and a high density of obstacles, resulting in a more complex path.

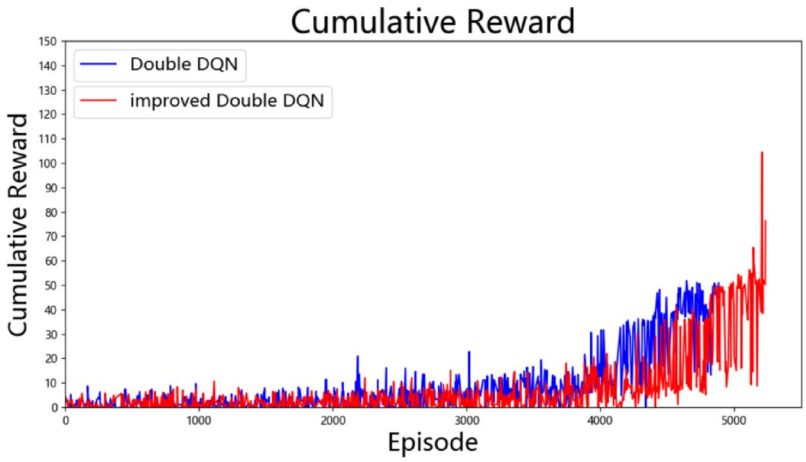


Fig. 11. Comparison of cumulative reward in "Long Corridor".

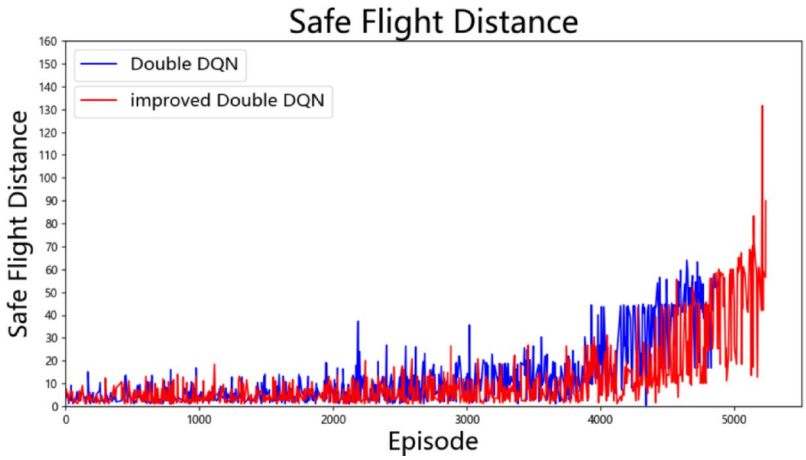


Fig. 12. Comparison of safe flight distance in "Long Corridor".

	Double DQN	Improved Double DQN	Improvement
Average cumulative reward	17.66	21.7	22.88%
Maximum cumulative reward	51.81	104.43	101.56%
Average safe flight distance(m)	22.7	27.96	23.17%
Maximum safe flight distance(m)	64	131.6	105.62%

Table 2. Algorithm result index comparison in "long corridor".

After performing 150,000 iterations with both the improved Double DQN algorithm and the traditional Double DQN algorithm in the experimental environment, the training results are shown in Figs. 14 and 15, and Fig. 16. Figure 14 illustrates the memory usage of the improved and traditional Double DQN algorithms throughout the 150,000 iterations in the simulation environment. It can be observed that the improved algorithm is more memory-efficient, especially after the initial rapid increase in memory usage, where it stabilizes at a level slightly lower than that of the traditional algorithm. Figures 15 and 16 depict the changes in cumulative rewards and flight distances for both algorithms during the experiment. The improved Double DQN shows better performance in both flight distance and cumulative rewards, particularly in the later stages of the episodes. It is noticeable from the figures that significant improvements in reward values and safe flight distances for both algorithms start after approximately 3,000 and 4,000 episodes, respectively. To accurately reflect the algorithms' performance in their stable states and avoid the influence of the early unstable phase on the evaluation results, we selected data from episode 3,000 to the last for the traditional algorithm and from episode 4,000 to the last for the improved algorithm to calculate the average cumulative rewards and flight distances per episode. Additionally, to mitigate the impact of outliers on the statistical results, we removed the top 5% and bottom

Fig. 13. The environment map of "Compact Living Room".

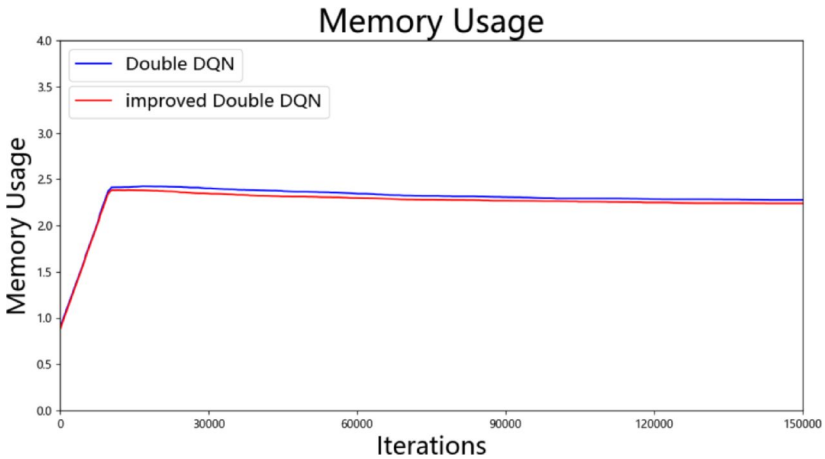


Fig. 14. Comparison of memory usage in "Compact Living Room".

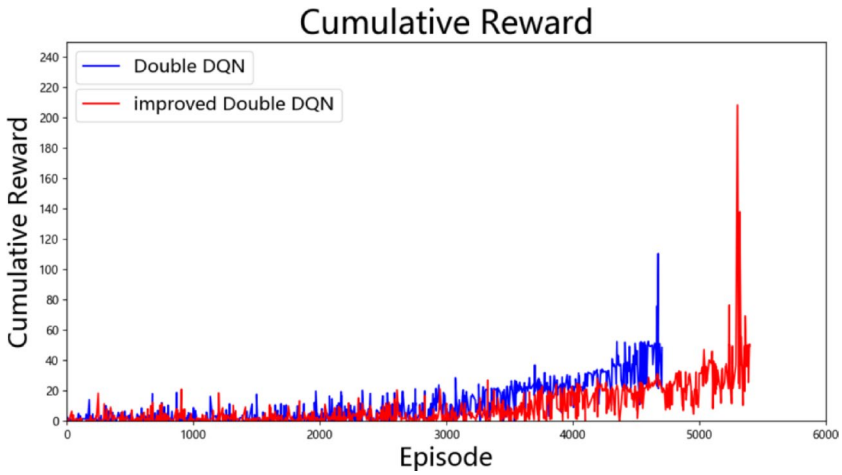


Fig. 15. Comparison of cumulative reward in "Compact Living Room".

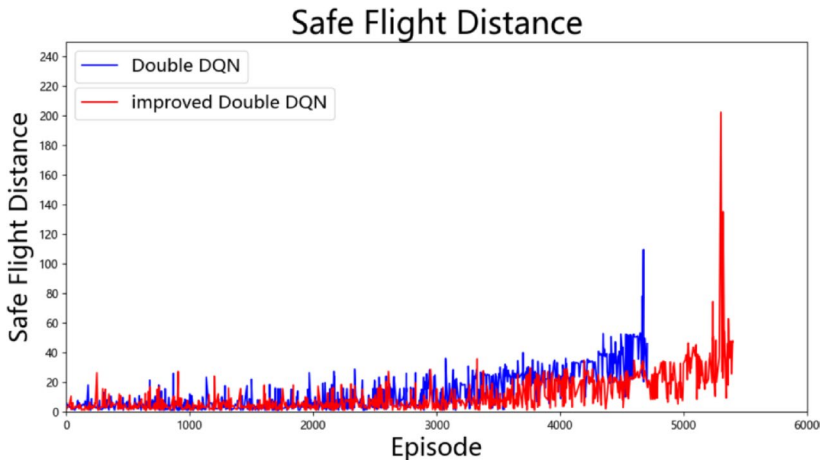


Fig. 16. Comparison of safe flight distance in "Compact Living Room".

	Double DQN	Improved Double DQN	Improvement
Average cumulative reward	22.2	22.79	2.66%
Maximum cumulative reward	110.37	208.47	88.77%
Average safe flight distance(m)	23.92	24.41	2.05%
Maximum safe flight distance(m)	109.6	202.4	84.68%

Table 3. Algorithm result index comparison in "compact living room".

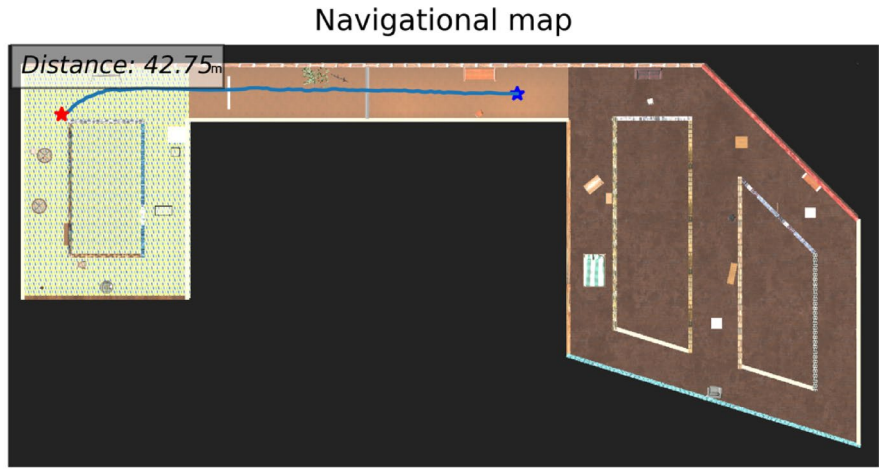
5% of values before the calculations. The experimental results, as shown in Table 3, indicate that the improved algorithm achieved a 2.7% increase in average cumulative rewards, an 88.8% increase in maximum cumulative rewards, a 2.1% increase in average flight distance, and an 84.7% increase in maximum flight distance compared to the traditional Double DQN algorithm. These results demonstrate that the improved Double DQN algorithm significantly outperforms the traditional one in terms of cumulative rewards and flight distances during the stable phase, indicating better performance and higher stability in the experimental environment. These findings confirm that the improved Double DQN algorithm surpasses the original in the stable phases of cumulative rewards and flight distances, proving its superior performance in complex indoor drone obstacle avoidance.

Experimental validation

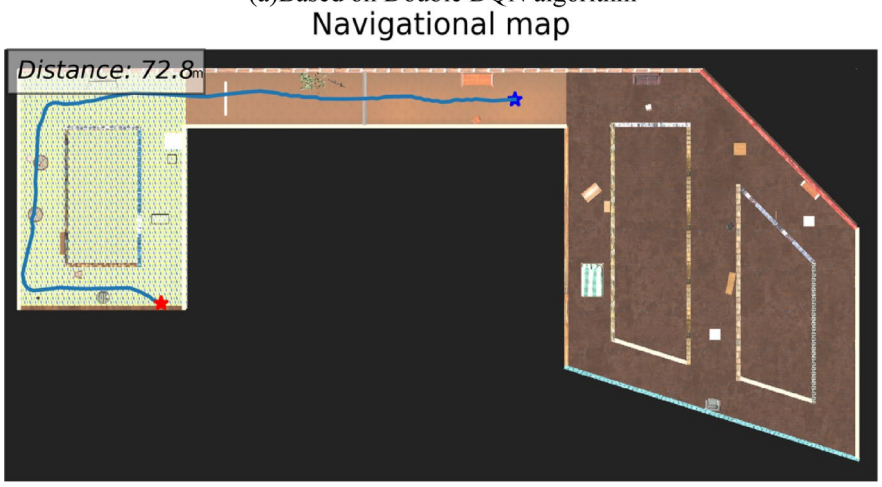
To thoroughly assess the performance of the improved Double DQN algorithm in UAV indoor autonomous obstacle avoidance, this paper designed an experimental validation phase. Specifically, the experiments used models trained with 150,000 iterations for both the traditional algorithm and the improved algorithm, and conducted inference tests in two indoor environments: "Long Corridor" and "Compact Living Room", to compare and analyze their performance. The actual safe flight distances obtained in "Long Corridor" are shown in Fig. 17, while those in "Compact Living Room" are shown in Fig. 18. The data obtained were then compared, with the experimental results presented in Tables 4 and 5. The improved Double DQN algorithm demonstrated significant performance improvements in both test environments. In "Long Corridor" environment, the improved algorithm increased the flight distance from 42.75 m to 72.8 m, achieving a performance improvement of 70.3%. Additionally, the processing time per meter decreased from 0.23 s to 0.21 s, representing an 8.7% improvement. In the "Compact Living Room" environment, the improved algorithm also showed excellent performance, increasing the flight distance from 52.35 m to 81.1 m, with a performance improvement of 54.9%. The processing time per meter decreased from 0.27 s to 0.26 s, indicating a 4.01% enhancement. These results demonstrate that the improved Double DQN algorithm excels in UAV indoor obstacle avoidance tasks, significantly increasing flight distance and enhancing the efficiency and accuracy of path planning in both relatively simple and more complex indoor environments.

Conclusions

Aiming to improve the insufficient autonomous obstacle avoidance performance of UAVs in complex indoor environments, this paper presents an improved Double DQN algorithm. By introducing the LSTM network, noise layer, and prioritized experience replay module, as well as designing a strategy for dynamically adjusting the exploration rate, the algorithm significantly enhances the obstacle avoidance performance of UAVs in indoor environments. The experimental outcomes indicate that the improved Double DQN algorithm outperforms the traditional Double DQN algorithm in terms of cumulative rewards and flight distance, effectively improving the autonomous obstacle avoidance capabilities of UAVs in complex indoor environments.



(a)Based on Double DQN algorithm



(b)Based on improved Double DQN algorithm

Fig. 17. Safe flight distance comparison in "Long Corridor".

Although the effectiveness of the improved Double DQN algorithm proposed in this paper has been validated through simulation analysis, it has not yet been extensively verified in practical applications. The following research directions are significant for future work:

1. Conduct field tests on actual drone platforms, such as applying it to intelligent transportation management, to verify the performance and robustness of the improved Double DQN algorithm in real-world scenarios.
2. Further optimize algorithm parameters and structures using simulation environments, and explore more advanced deep learning technologies to enhance the autonomous obstacle avoidance capabilities of drones.
3. By integrating multimodal human-machine (HM) interaction, the improved Double DQN algorithm will enhance the drone's responsiveness to human commands, optimizing the efficiency of human-machine collaboration.
4. Study the adaptability of the algorithm in dynamically changing environments to improve its generalization and robustness.

(a)Based on Double DQN algorithm

(b)Based on improved Double DQN algorithm

Fig. 18. Safe flight distance comparison in "Compact Living Room".

	Double DQN	Improved Double DQN	Improvement
Safe flight distance(m)	42.75	72.8	70.32%
Processing time(s)	9.82	15.26	-
Processing time per meter(s/m)	0.23	0.21	8.7%

Table 4. Validation result comparison in "long corridor".

	Double DQN	Improved Double DQN	Improvement
Safe flight distance(m)	52.35	81.8	54.9%
Processing time(s)	14.32	21.38	-
Processing time per meter(s/m)	0.27	0.26	4.0%

Table 5. Validation result comparison in "compact living room".

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Received: 10 March 2025; Accepted: 13 May 2025
Published online: 01 August 2025

References

1. Shandilya, K. S. et al. AI, Cybersecurity and Data Science for Drone and Unmanned Aerial Vehicles:Real-Life Applications and Case Studies[M].CRC Press:2025-02-21.
2. Sharma, S. *Drone Development from Concept To Flight: Design, Assemble, and Discover the Applications of Unmanned Aerial vehicles[M]* (Packt Publishing Ltd, 2024).
3. Wang, H. UAV obstacle avoidance with PID control based on improved sparrow search algorithm[C]//Fourth International Conference on Signal Processing and Machine Learning (CONF-SPML 2024). SPIE, 13077: 252–263. (2024).
4. Nhouchi, A. et al. A real-time A* algorithm for trajectories generation and collision avoidance in uncertain environments for assembly applications[J]. *Comput. Ind. Eng.*, : 110959. (2025).
5. Fu, H., Shao, G. & ,Zhao, L. Global path planning of un-manned ship island region based on improved A* al-gorithm[J].Journal of Physics: Conference Series,2858(1):012021–012021. (2024).
6. Wang, Y. et al. Hybrid path planning for USV with kinematic constraints and COLREGS based on improved APF-RRT and DWA[J]. *Ocean Eng.* **318**, 120128 (2025).
7. Guo, S. et al. DBVSB-P-RRT*: A path planning algorithm for mobile robot with high environmental adaptability and ultra-high speed planning[J]. *Expert Syst. Appl.* **266**, 126123 (2025).
8. Langxiong, G. et al. Intelligent ship path planning based on improved artificial potential field in narrow inland waterways[J]. *Ocean Eng.* **317**, 119928 (2025).
9. Hu, J. B. et al. Path planning OfUrban UAV based on Multi-objective particle swarm optimization [J]. *Aviat. Comput. Technol.* **54** (05), 38–42 (2024).
10. Sahal, M., Maynad, V. C. & Bilfaqih, Y. Cooperative formation and obstacle avoidance control for Multi-UAV based on guidance route and artificial potential Field[J]. *J. Rob. Control (JRC).* **5** (6), 1772–1783 (2024).
11. Sahal, M. et al. Strategic control in cooperative Multi-Agent moving source seeking with Obstacles Avoidance[J]. *Int. J. Intell. Eng. Syst.*, **16**(5). (2023).
12. Yang, F. et al. Obstacle avoidance path planning for UAV based on improved RRT algorithm[J]. *Discrete Dynamics Nat. Soc.* **2022** (1), 4544499 (2022).
13. Zhao, Z., Wan, Y. & ,Chen, Y. Deep reinforcement Lear-ning-Driven collaborative Rounding-Up for multiple unmanned aerial vehicles in obstacle Environments[J].Drones,2024,**8**(9):464–464 .
14. Xiu, C. J. Autonomous driving Decision-Making method based on improved DQN [J]. *Times Automot.*, (20): 28–31. (2024).
15. Yao, G. et al. Improved SARSA and DQN algorithms for reinforcement learning[J]. *Theor. Comput. Sci.* **1027**, 115025 (2025).
16. Li Ming, Y. et al. Desert r-obot path planning based on deep reinforcement lear-ning [J/OL]. *J. Syst. Simul.* : 1–9 [2024-04-07].
17. Li, J. et al. A Multi-Area task Pa-th-Planning algorithm for agricultural drones basedon improved double deep Q-Learning Net[J]. Agric-ulture,2024,**14**(8):1294–1294 .
18. Zhao, Y. et al. *Robot Automatic Path Planning by Avoiding Obstacle Using Double Deep Q Networks on the Testbed[J]*19–20 (한국콘텐츠학회 ICCC 논문집, 2022).
19. Tang, C. et al. 3D path planning ofunmanned ground vehicles based on improved DDQ-N[J]. *J. Supercomputing 2024*, **81**(1):276–276 .
20. Yao, W. et al. Study on UAV o-bstacle avoidance algorithm based on deep recurrentdouble Q network[J].Xibei Gongye Daxue Xuebao/Jo-urnal of Northwestern polytechnical university,2022,**40**(5):970–979 .
21. Singh, A. et al. A Vision-Based Bio-Inspired reinforcement learning algorithms for manipulator obstacle Avoidance[J]. *Electronics* **11** (21), 3636 (2022).
22. Sahal, M. et al. Obstacle Avoidance System on Autonomous Car Using D3QN[C]//2023 14th International Conference on Information & Communication Technology and System (ICTS). IEEE, : 199–204. (2023).
23. Lu, B. et al. *Enhanced LSTM-DQN Algorithm for a two-player zero-sum Game in three-dimensional space[J]*182798–2812 (IET Control Theory & Applications, 2024). 18.
24. Tian, X. et al. LSTM & attention-based meta-reinforcement learning for trajectory tracking of underwater gliders with varying buoyancy loss and current disturbance[J]. *Ocean Eng.* **326**, 120906 (2025).
25. Zhang, B. et al. *A NoisyNet Deep Reinforcement Learning Method for Frequency Regulation in Power systems[J]*183042–3051 (IET Generation, Transmission & Distribution, 2024). 19.
26. Wang, L. & Wang, X. Enhanced deep reinforcement learning strategy for energy management in Plug-in hybrid electric vehicles with entropy regularization and prioritized experience Replay[J].Energy Engine-ering,2024,**121**(12):3953–3979 .

27. Ozsoydan, F. B. Reinforcement learning enhanced swarm intelligence and trajectory-based algorithms for parallel machine scheduling problems[J]. *Comput. Ind. Eng.*, : 110948. (2025).
28. Cui, Q., Feng, G. & Xu, X. *Q-Learning-Based Robust Control for Nonlinear Systems with Mismatched Perturbations[J]* (IEEE Transactions on Neural Networks and Learning Systems, 2025).
29. Cui, M. et al. Path planning for mobile robot based on improved ant colony Q-learning algorithm[J]. *Int. J. Interact. Des. Manuf. (IJIDeM)*. **19** (4), 3069–3087 (2025).
30. Jiang, Z., Zhang, H. & Xiao, Y. Data-based discrete-time two-player zero-sum delayed game via policy iteration Q-learning Method[J]. *Neurocomputing* **631**, 129709 (2025).
31. AbdelAziz, M. N. et al. Deep Q-Network (DQN) model for disease prediction using electronic health records (EHRs)[J]. Sci,2025,**7**(1):14–14 .
32. Lu, X. et al. DQN-Based automatic emergency collision avoidance control considering driver Style[J]. *Int. J. Autom. Technol.*, : 1–12. (2025).
33. Lu, S. et al. A Study on the Impact of Obstacle Size on Training Models Based on DQN and DDQN[C]//ITM Web of Conferences. EDP Sciences, 73: 01004. (2025).
34. Xu, H. & Zhu, D. Multiple unmanned aerial vehicle collaborative target search by DRL: A DQN-Based Multi-Agent partially observable Method[J]. (2025). Drones (2504-446X), **9**(1).
35. Zhuang, X. & Tong, X. A dynamic algorithm for trust inference based on double DQN in the internet of things[J]. *Digit. Commun. Networks*. **10** (4), 1024–1034 (2024).
36. Ding, Z. et al. A modular robotic arm configuration design method based on double DQN with prioritized experience Replay[J]. *Symmetry* **16** (6), 714 (2024).
37. Workneh, A. D. & Gmira, M. Learning to schedule (L2S): adaptive job shop scheduling using double deep Q network[J]. *Smart Sci.* **11** (3), 409–423 (2023).
38. Yu, Y. et al. Obstacle avoidance method based on double DQN for agricultural robots[J]. *Comput. Electron. Agric.* **204**, 107546 (2023).
39. Zheng, Y. et al. Pri-DDQN: learning adaptive traffic signal control strategy through a hybrid agent[J]. *Complex. Intell. Syst.* **11** (1), 47 (2025).

Acknowledgements
This work was supported by the Taishan Scholar Project of Shandong Province(tshw201502042) and Major Innovation Engineering Project of Shandong Province(2017 CXGC0607).

Author contributions
Ruiqi Yu contributed to conceptualization, methodology, software, data analysis, and writing. Qingdang Li and Tingting Wu assisted with data curation and editing. Jiewei Ji validated the methodology. Jian Mao, Shun Liu, and Zhen Sun supervised the project and reviewed the manuscript. All authors approved the submitted version.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to Z.S.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025